# Lecture 07: Text Processing

CE350: Linux Utilities and Shell Scripting

Çağlar Durmaz

# Previously on CE350

- sed and awk

# Text Processing

- All UNIX-like operating systems rely heavily on text files for several types of data storage.
- So it makes sense that there are many tools for manipulating text.
- Let’s see the programs that are used to “slice and dice” text.

# Text Processing

- Here are a list of them. Do they look familiar?
  - cat
  - sort
  - uniq
  - shuf
  - cut
  - paste
  - join
  - comm
  - diff
  - patch
  - tr

# Text Processing

- Obviously, text is used for a lot of purposes.
- **Documents**: Many people write documents using plain text formats.
- It makes sense that a small text file could be useful for simple notes.
- It is also possible to write large documents in text format.
- One popular approach is to write a large document in a text format and then use a markup language to describe the formatting of the finished document.
- Many scientific papers are written using LaTeX; UNIX-based text processing systems were among the first systems that supported advanced typographical layout needed by writers in technical disciplines.

# Text Processing

- **Email**: is an intrinsically text-based medium. Even non-text attachments are converted into a text representation for transmission.
- We can see this for ourselves by downloading an email message and then viewing it in less.
- It begins with a **header** that describes the source of the message, and the processing it received during its journey, followed by the **body** of the message.
- We (Umut, İlker, and Kaya) always ask each other for help, so, here is the source of an email conversation among us.

koguz — pi@zeytin: ~ — ssh pi@zeytin — 92×42

```
Return-Path: ilker.korkmaz@izmirekonomi.edu.tr
Received: from zcsper.izmirekonomi.edu.tr (LHLO zcsper.izmirekonomi.edu.tr)
 (10.0.2.22) by zcsper.izmirekonomi.edu.tr with LMTP; Thu, 16 Feb 2017
 14:55:47 +0200 (EET)
Received: from localhost (localhost [127.0.0.1])
        by zcsper.izmirekonomi.edu.tr (Postfix) with ESMTP id CC17FBA96B;
        Thu, 16 Feb 2017 15:55:47 +0300 (+03)
Received: from zcsper.izmirekonomi.edu.tr ([127.0.0.1])
        by localhost (zcsper.izmirekonomi.edu.tr [127.0.0.1]) (amavisd-new, port 10032)
        with ESMTP id mvaW8UWqGBTB; Thu, 16 Feb 2017 15:55:45 +0300 (+03)
Received: from localhost (localhost [127.0.0.1])
        by zcsper.izmirekonomi.edu.tr (Postfix) with ESMTP id 03303BAAB4;
        Thu, 16 Feb 2017 15:55:45 +0300 (+03)
X-Virus-Scanned: amavisd-new at izmirekonomi.edu.tr
Received: from zcsper.izmirekonomi.edu.tr ([127.0.0.1])
        by localhost (zcsper.izmirekonomi.edu.tr [127.0.0.1]) (amavisd-new, port 10026)
        with ESMTP id N3aOowUNhfBr; Thu, 16 Feb 2017 15:55:44 +0300 (+03)
Received: from zcsper.izmirekonomi.edu.tr (zcsper.izmirekonomi.edu.tr [10.0.2.22])
        by zcsper.izmirekonomi.edu.tr (Postfix) with ESMTP id D1F2DBA96B;
        Thu, 16 Feb 2017 15:55:44 +0300 (+03)
Date: Thu, 16 Feb 2017 14:55:44 +0200 (EET)
From: Ilker Korkmaz <ilker.korkmaz@izmirekonomi.edu.tr>
To: Kaya =?iso-8859-9?Q?O=F0uz?= <kaya.oguz@ieu.edu.tr>
Cc: Umut Avci <umut.avci@izmirekonomi.edu.tr>
Message-ID: <646437116.10028421.1487249744720.JavaMail.zimbra@izmirekonomi.edu.tr>
In-Reply-To: <c9487872-7945-9001-2586-d047a098ba34@ieu.edu.tr>
References: <SEMA-CR-1-49VC43O@bmsmail2.ieee.org> <66d9ad33-fa40-fb83-0379-16f9dbc7f8fb@ieu.
edu.tr> <1879563364.9953352.1487221865324.JavaMail.zimbra@izmirekonomi.edu.tr> <1085426976.9
955178.1487224272872.JavaMail.zimbra@izmirekonomi.edu.tr> <c9487872-7945-9001-2586-d047a098b
a34@ieu.edu.tr>
Subject: Re: Call for Papers: AIPR2017 Conference in Lodz, Poland
MIME-Version: 1.0
Content-Type: text/plain; charset=ISO-8859-9
Content-Transfer-Encoding: quoted-printable
X-Originating-IP: [172.16.1.218]
X-Mailer: Zimbra 8.0.7_GA_6021 (ZimbraWebClient - FF51 (Win)/8.0.7_GA_6021)
Thread-Topic: Call for Papers: AIPR2017 Conference in Lodz, Poland
Thread-Index: F2rJNgNxapmwX3n4eF/7MMYjAYAusA==

:) hey gidi g=FCnler heeey !
```

# Text Processing

- **Printer Output:** On UNIX-like systems, output destined for a printer is sent as plain text or, if the page contains graphics, is converted into a text format **page description language**, known as **PostScript**, which is then sent to a program that generates the graphic dots to be printed.
- **Program source code:** Many of the command line programs found on UNIX-like systems were created to support system administration and software development, and text processing programs are no exception.
- Many of them are designed to solve software development problems.
- The source code is always in text format.

# Text Processing: cat

- The **cat** program has a number of interesting options.
- The -A option displays nonprinting characters in the text.
- There may be times when we want to know if control characters are embedded in our otherwise visible text.
- The most common of these are tab characters and carriage returns, often present as end-of-line characters.

```
pi@zeytin:~ $ vim
pi@zeytin:~ $ cat cat00
I will leave a lot of spaces at the end of this line, intentionally.

pi@zeytin:~ $ cat -A cat00
I will leave a lot of spaces at the end of this line, intentionally.    $
$
pi@zeytin:~ $
```

# Text Processing: cat

- I assume you can tell what $ means, in this case.
- Let me try a tab.

```
pi@zeytin:~ $ vim
pi@zeytin:~ $ cat cat00
I will leave a lot of spaces at the end of this line, intentionally.

pi@zeytin:~ $ cat -A cat00
I will leave a lot of spaces at the end of this line, intentionally.    $
$
pi@zeytin:~ $
```

# Text Processing: cat

- The ^I means a tab. You didn't expect a \t, did you?
- The caret shows the Control key on the keyboard. So Control+I is the tab character.
- On UNIX, the line endings are \n, however, on Windows it is \r\n, where \r is the return carriage.
- At first I've tried to find a text file written on Windows.
- Then, I said, why not write a C program that can output \r\n?

```
pi@zeytin:~ $ cat cat00
I will leave a lot of spaces at the end of this line, intentionally.
        a tab           another

pi@zeytin:~ $ cat -A cat00
I will leave a lot of spaces at the end of this line, intentionally.    $
^Ia tab^I^Ianother$
$
```

```
pi@zeytin:~ $ vim win.c
pi@zeytin:~ $ gcc win.c -o win
pi@zeytin:~ $ ./win
pi@zeytin:~ $ cat -A win.txt
Hello world^M$
This is a test^M$
pi@zeytin:~ $
```

# Text Processing: cat

- The ^I means a tab. You didn't expect a \t, did you?
- The caret shows the Control key on the keyboard. So Control+I is the tab character.
- On UNIX, the line endings are \n, however, on Windows it is \r\n, where \r is the return carriage.
- At first I've tried to find a text file written on Windows.
- Then, I said, why not write a C program that can output \r\n?

```c
#include <stdio.h>

int main() {
        FILE* f = fopen("win.txt", "w");
        fprintf(f, "Hello world\r\nThis is a test\r\n");
        fclose(f);
        return 0;
}
```

# Text Processing: cat

- Another option for cat, is the -n, where it prints out the line numbers.

```
pi@zeytin:~ $ cat -An cat00
     1  I will leave a lot of spaces at the end of this line, intentionally.    $
     2  ^Ia tab^I^Ianother$
     3  $
```

# Text Processing: cat

- The real purpose of cat is to concatenate text.
- We never used it, but it does not mean that we will never need or use it.
- Here is a demonstration.
- Let’s concatenate the following three files into one file by using redirection: uptime, win.c, and cat00.

```
pi@zeytin:~ $ cat uptime win.c cat00 > concatenated
pi@zeytin:~ $ cat concatenated
 23:19:15 up 14 days,  9:24,  3 users,  load average: 0.02, 0.02, 0.03
 15:16:18 up 18 days,  1:21,  3 users,  load average: 0.08, 0.06, 0.01
 21:28:13 up 19 days,  7:33,  4 users,  load average: 0.42, 0.29, 0.12
 14:46:42 up 9 days, 15:53,  3 users,  load average: 0.10, 0.04, 0.01
 19:44:48 up 23 days, 20:51,  2 users,  load average: 0.00, 0.01, 0.00
 08:47:57 up 24 days,  9:55,  3 users,  load average: 0.02, 0.04, 0.00
 00:12:27 up 30 days,  1:19,  3 users,  load average: 0.00, 0.00, 0.00
 09:31:04 up 31 days, 10:38,  3 users,  load average: 0.06, 0.05, 0.00
 14:23:24 up 37 days, 15:30,  3 users,  load average: 0.05, 0.02, 0.00
 17:19:26 up 38 days, 18:26,  3 users,  load average: 0.20, 0.12, 0.03
 00:07:34 up 39 days,  1:14,  3 users,  load average: 0.07, 0.08, 0.03
#include <stdio.h>

int main() {
        FILE* f = fopen("win.txt", "w");
        fprintf(f, "Hello world\r\nThis is a test\r\n");
        fclose(f);
        return 0;
}


I will leave a lot of spaces at the end of this line, intentionally.
        a tab           another
```

# Text Processing: sort

- The **sort** program sorts the contents of standard input, or one or more files specified on the command line.
- It can accept multiple files on the command line as arguments, so it is possible to merge multiple files into a single sorted whole.
- Let’s try to use an output of a command to sort, instead of a file.
- The program **du**, disk usage, gives the disk usage of files and directories.
- Its -s option summarizes the output.
- I’ll find the disk usage of the files in /usr/share, however, there are too many.
- Pipe this output to **head** program, which, by default, prints the first 10 lines.
- Its cousin, **tail**, prints the last 10 lines by default.

# Text Processing: sort

- Clearly, du works with the files in alphabetical order.
- What if I want to see the largest files in this directory?
- Then, let me sort this output.

```
pi@zeytin:~ $ du -s /usr/share/* | head
11564   /usr/share/abiword-3.0
252     /usr/share/aclocal
8       /usr/share/adduser
184     /usr/share/alacarte
680     /usr/share/alsa
64      /usr/share/appdata
16      /usr/share/application-registry
384     /usr/share/applications
68      /usr/share/apport
12      /usr/share/apps
```

# Text Processing: sort

- It didn’t work, because sort assumed that the numbers are strings.
- As a string, 1016 comes before 112.
- We can use the -n option of sort to fix this.

```
pi@zeytin:~ $ du -s /usr/share/* | sort | head
100     /usr/share/apt-listchanges
1016    /usr/share/lintian
1032    /usr/share/hal
108     /usr/share/vnc
109328  /usr/share/doc
112     /usr/share/consoletrans
112     /usr/share/lxappearance
11564   /usr/share/abiword-3.0
116     /usr/share/libvisual-plugins-0.4
120     /usr/share/doc-base
```

# Text Processing: sort

- Another problem.
- Since it sorts the numbers, now I have the list in ascending order, the smallest ones first.
- What can I do?

```
pi@zeytin:~ $ du -s /usr/share/* | sort -n | head
4       /usr/share/misc
4       /usr/share/pkg-config-crosswrapper
4       /usr/share/rubygems-integration
4       /usr/share/squeak
8       /usr/share/adduser
8       /usr/share/debian-reference-common
8       /usr/share/debianutils
8       /usr/share/dict
8       /usr/share/enchant
8       /usr/share/gksu
```

# Text Processing: sort

- I can use the -r option for sort, which reverses the list, and it uses a descending order.
- I can also use the **tail** program, instead of **head**, right?
- Do they print the same thing?

```
pi@zeytin:~ $ du -s /usr/share/* | sort -n | head
4       /usr/share/misc
4       /usr/share/pkg-config-crosswrapper
4       /usr/share/rubygems-integration
4       /usr/share/squeak
8       /usr/share/adduser
8       /usr/share/debian-reference-common
8       /usr/share/debianutils
8       /usr/share/dict
8       /usr/share/enchant
8       /usr/share/gksu
```

# Text Processing: sort

- Isn’t symmetry a beautiful thing?
- Choose the one that suits your needs.

```
pi@zeytin:~ $ du -s /usr/share/* | sort -nr | head
222928  /usr/share/locale
109328  /usr/share/doc
86020   /usr/share/scratch
84508   /usr/share/icons
42328   /usr/share/fonts
32852   /usr/share/midi
29304   /usr/share/vim
27468   /usr/share/pixel-wallpaper
24016   /usr/share/man
19396   /usr/share/perl
pi@zeytin:~ $ du -s /usr/share/* | sort -n | tail
19396   /usr/share/perl
24016   /usr/share/man
27468   /usr/share/pixel-wallpaper
29304   /usr/share/vim
32852   /usr/share/midi
42328   /usr/share/fonts
84508   /usr/share/icons
86020   /usr/share/scratch
109328  /usr/share/doc
222928  /usr/share/locale
```

# Text Processing: sort

- You may ask: can we use it with the output of **ls**?
- The long option, -l, for ls actually prints the size of each file, but the size field is not at the beginning of the line.
- It's in the…

```
-rw-r--r-- 1 pi pi      48 Apr  1 21:41 toc.book.tmp
-rwxr-xr-x 1 pi pi     154 Apr  1 21:41 toc.sh
-rw-r--r-- 1 pi pi     780 Apr  2 00:07 uptime
drwxr-xr-x 2 pi pi    4096 Nov 25 21:10 Videos
```

  fifth field.
- What can I do?

# Text Processing: sort

- At this moment, I might have heard some answers that contain a solution with a particular command we've covered; awk.
- There is another option; the -k option of the sort command.
- This option sorts based on a key field located from field1 to field2, rather than the entire line.

```
pi@zeytin:~ $ ls -l | sort -nr -k 5 | head
-rw-r--r-- 1 pi pi 1417791 Jan 24 18:13 2017-01-24-171300_1824x984_scrot.png
-rw-r--r-- 1 pi pi  536098 Feb  7 11:35 OlympicAthletes.csv
-rw-r--r-- 1 pi pi    7320 Apr  2 12:31 test.txt
-rw------- 1 pi pi    6144 Apr  2 12:13 ilker.umut
-rwxr-xr-x 1 pi pi    5980 Apr  2 12:34 win
drwxr-xr-x 7 pi pi    4096 Mar 24 14:34 sed
drwxr-xr-x 5 pi pi    4096 Jan 25 23:25 Documents
drwxr-xr-x 3 pi pi    4096 Mar  6 11:00 development
drwxr-xr-x 2 pi pi    4096 Nov 25 21:10 Videos
drwxr-xr-x 2 pi pi    4096 Nov 25 21:10 Templates
```

# Text Processing: sort

- Another example using the -k option of the sort command.

Terminal - ilker@debian: ~

File Edit View Terminal Tabs Help

```
ilker@debian:~$ cat test.txt
1 ilker korkmaz 80
2 kaya oguz     85
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ cat -A test.txt
1 ilker korkmaz 80$
2 kaya oguz^I85$
3 umut avci^I85$
4 damla oguz^I99$
ilker@debian:~$ sort test.txt
1 ilker korkmaz 80
2 kaya oguz     85
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ sort -k 1 test.txt
1 ilker korkmaz 80
2 kaya oguz     85
3 umut avci     85
4 damla oguz    99
ilker@debian:~$
```

# Text Processing: sort

- Another example using the -k option of the sort command.

```
ilker@debian:~$ cat test.txt
1 ilker korkmaz 80
2 kaya oguz     85
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ sort -k 2 test.txt
4 damla oguz    99
1 ilker korkmaz 80
2 kaya oguz     85
3 umut avci     85
ilker@debian:~$ sort -k 3 test.txt
3 umut avci     85
1 ilker korkmaz 80
2 kaya oguz     85
4 damla oguz    99
ilker@debian:~$ sort -k 4 test.txt
1 ilker korkmaz 80
2 kaya oguz     85
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ sort -k 4n test.txt
1 ilker korkmaz 80
2 kaya oguz     85
3 umut avci     85
4 damla oguz    99
ilker@debian:~$
```

# Text Processing: sort

- Using the -o option of the sort command, we can redirect the output to a file.

Terminal - ilker@debian: ~

File Edit View Terminal Tabs Help

```
ilker@debian:~$ cat test2.txt
1 ilker korkmaz 80
2 kaya oguz     58
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ sort -k 4n test2.txt > newOutput.txt
ilker@debian:~$ sort -k 4n -o newOutput2.txt test2.txt
ilker@debian:~$ sort -o newOutput3.txt -k 4n test2.txt
ilker@debian:~$ cat newOutput.txt
2 kaya oguz     58
1 ilker korkmaz 80
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ cat newOutput2.txt
2 kaya oguz     58
1 ilker korkmaz 80
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ cat newOutput3.txt
2 kaya oguz     58
1 ilker korkmaz 80
3 umut avci     85
4 damla oguz    99
ilker@debian:~$
```

# Text Processing: sort

- We can actually do more with the -k option.
- Consider the distros file.
- If we sort it, using **sort**, then it will sort the lines alphabetically.
- If we use the -k option, and sort according to the version numbers, then we will lose the ordering in the distro name.
- So, what I want is: sort with the first field first, then, sort according to the second field, but sort it numerically.

```
pi@zeytin:~ $ cat distros.txt
SUSE    10.2    12/07/2006
Fedora  10      11/25/2008
SUSE    11.0    06/19/2008
Ubuntu  8.04    04/24/2008
Fedora  8       11/08/2007
SUSE    10.3    10/04/2007
Ubuntu  6.10    10/26/2006
Fedora  7       05/31/2007
Ubuntu  7.10    10/18/2007
Ubuntu  7.04    04/19/2007
SUSE    10.1    05/11/2006
Fedora  6       10/24/2006
Fedora  9       05/13/2008
Ubuntu  6.06    06/01/2006
Ubuntu  8.10    10/30/2008
Fedora  5       03/20/2006
```

# Text Processing: sort

```
pi@zeytin:~ $ sort --key=1,1 --key=2n distros.txt
Fedora  5       03/20/2006
Fedora  6       10/24/2006
Fedora  7       05/31/2007
Fedora  8       11/08/2007
Fedora  9       05/13/2008
Fedora  10      11/25/2008
SUSE    10.1    05/11/2006
SUSE    10.2    12/07/2006
SUSE    10.3    10/04/2007
SUSE    11.0    06/19/2008
Ubuntu  6.06    06/01/2006
Ubuntu  6.10    10/26/2006
Ubuntu  7.04    04/19/2007
Ubuntu  7.10    10/18/2007
Ubuntu  8.04    04/24/2008
Ubuntu  8.10    10/30/2008
```

- We could have written -k 1,1 -k 2n, but for the first time, I wanted it to improve readability.
- We specify a range of keys in the first key.
- Since we wanted to limit the sort to just the first field, we specified 1,1, which means “start at field one, stop at field one.”
- In the second instance, we specified 2n, which means that field 2 is the sort key, and that the sort should be numeric.

# Text Processing: sort

- We also have a date field; 19/01/2010, but it looks impossible to work with.
- Fortunately, **sort** provides a way.
- The key option allows specification of **offsets** within fields, so we can define keys within fields.

# Text Processing: sort

- We also have a date field; 19/01/2010, but it looks impossible to work with.
- Fortunately, **sort** provides a way.
- The key option allows specification of **offsets** within fields, so we can define keys within fields.

```
pi@zeytin:~ $ sort -k 3.7nbr -k 3.1nbr -k 3.4nbr distros.txt
Fedora  10      11/25/2008
Ubuntu  8.10    10/30/2008
SUSE    11.0    06/19/2008
Fedora  9       05/13/2008
Ubuntu  8.04    04/24/2008
Fedora  8       11/08/2007
Ubuntu  7.10    10/18/2007
SUSE    10.3    10/04/2007
Fedora  7       05/31/2007
Ubuntu  7.04    04/19/2007
```

# Text Processing: sort

- By specifying -k 3.7, we instruct **sort** to use a sort key that begins at the seventh character within the third field, the year.
- Likewise, we specify 3.1 and 3.4 for the month and day.
- We also add the n and r options to achieve a numeric sort.
- The b option is included to suppress leading spaces which vary from line to line.

```
pi@zeytin:~ $ sort -k 3.7nbr -k 3.1nbr -k 3.4nbr distros.txt
Fedora  10      11/25/2008
Ubuntu  8.10    10/30/2008
SUSE    11.0    06/19/2008
Fedora  9       05/13/2008
Ubuntu  8.04    04/24/2008
Fedora  8       11/08/2007
Ubuntu  7.10    10/18/2007
SUSE    10.3    10/04/2007
Fedora  7       05/31/2007
Ubuntu  7.04    04/19/2007
```

# Text Processing: sort

- The fields in some files are delimited with colons (:)
- For such cases, **sort** provides the -t option to define the field separator character.
- To sort the **passwd** file on the seventh field, which is the user's default shell, we could:

```
pi@zeytin:~ $ sort -t ':' -k 7 /etc/passwd | head
git:x:1001:1001:,,,:/home/git:/bin/bash
pi:x:1000:1000:,,,:/home/pi:/bin/bash
root:x:0:0:root:/root:/bin/bash
avahi:x:105:110:Avahi mDNS daemon,,,:/var/run/avahi-daemon:/bin/false
lightdm:x:109:114:Light Display Manager:/var/lib/lightdm:/bin/false
messagebus:x:104:109::/var/run/dbus:/bin/false
ntp:x:106:111::/home/ntp:/bin/false
pulse:x:110:116:PulseAudio daemon,,,:/var/run/pulse:/bin/false
```

# Text Processing: uniq

- Compared to **sort**, the **uniq** program is a lightweight.
- It performs seemingly trivial task.
- When given a sorted file, including the standard output, it removes any duplicate lines and sends the results to the standard output.
- It is often used in conjunction with **sort** to clean the output of duplicates.

# Text Processing: uniq

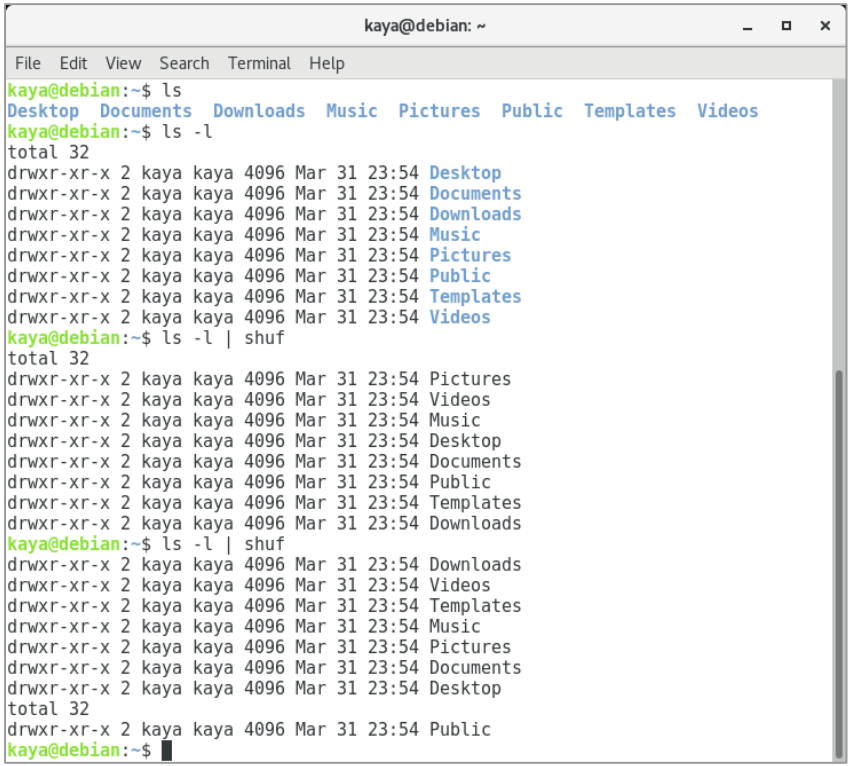
- The uniq program has interesting options.
- For example, -c option outputs a list of duplicate lines preceded by the number of times the line occurs.
- The -d option only outputs repeated lines, rather than unique lines.

```
pi@zeytin:~ $ awk '{ print $1 }' distros.txt | sort | uniq
Fedora
SUSE
Ubuntu
pi@zeytin:~ $ awk '{ print $1 }' distros.txt | sort | uniq -c
      6 Fedora
      4 SUSE
      6 Ubuntu
pi@zeytin:~ $ awk '{ print $1 }' distros.txt | sort | uniq -d
Fedora
SUSE
Ubuntu
```

# Text Processing: shuf

- If we can sort, then we should also be able to shuffle…
- The **shuf** command generates random permutations of input lines.
- So we can get randomization if we want to…

```
$ seq 1 10 | shuf | sort -n
```

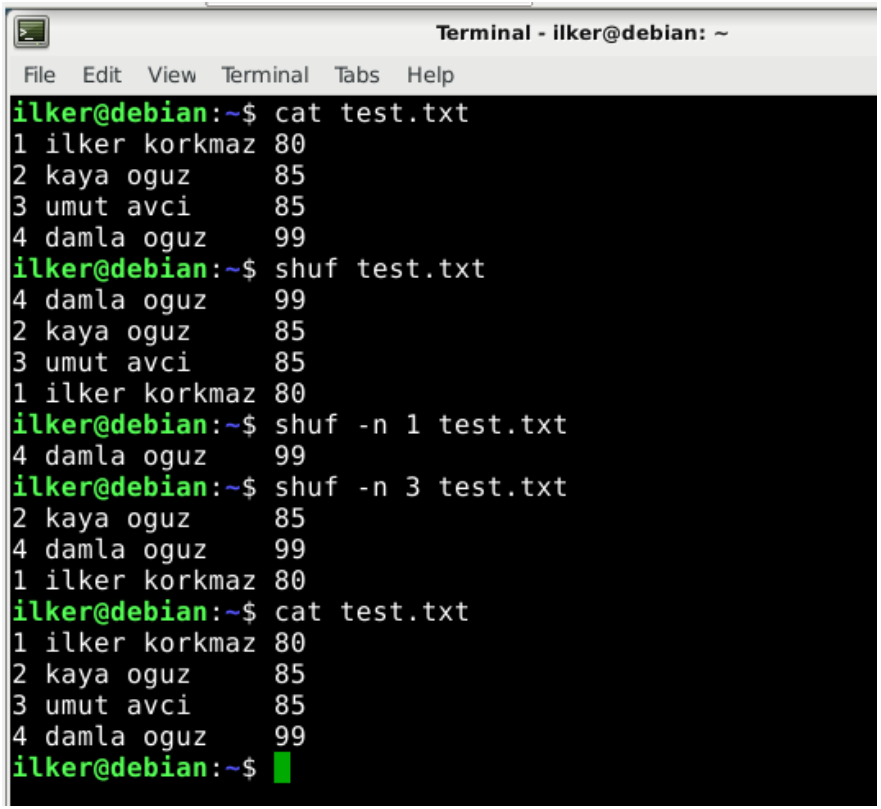


# Text Processing: shuf

- Using -n option with **shuf** command will pick line(s) randomly from input lines.

```
$ shuf -n 1 inputFile

$ shuf -n 3 inputFile
```



# Text Processing: cut

- The **cut** program is used to extract a section of text from a line, and output the extracted section to standard output.
- It can accept multiple file arguments or input from standard input.
- Specifying the section of the line to be extracted is somewhat awkward and is specified using the following options
  - The -c option works on a character list
  - The -f option works on a field list
  - The -d option changes the delimiter
  - The --complement option extracts the entire line of text, except for those portions specified.
- Let’s work on the distros again.

# Text Processing: cut

- The first command cuts the third field.
- The second command cuts the third field, which is then piped to another cut which works on the characters in the range 7 to 10.
- I have used **head** program in between pipes to print the first five lines only.

```
pi@zeytin:~ $ cut -f 3 distros.txt | head -n 5
12/07/2006
11/25/2008
06/19/2008
04/24/2008
11/08/2007
pi@zeytin:~ $ cut -f 3 distros.txt | head -n 5 | cut -c 7-10
2006
2008
2008
2008
2007
```

# Text Processing: paste

- If you cut something, then you probably will paste it.
- Let me cut from distros, then paste into a new file.

```
pi@zeytin:~ $ cut -f 3 distros.txt | cut -c 7-10 > distro-dates
pi@zeytin:~ $ cut -f 1,2 distros.txt > distro-versions
pi@zeytin:~ $ paste distro-dates distro-versions
2006    SUSE    10.2
2008    Fedora  10
2008    SUSE    11.0
2008    Ubuntu  8.04
2007    Fedora  8
2007    SUSE    10.3
2006    Ubuntu  6.10
2007    Fedora  7
2007    Ubuntu  7.10
2007    Ubuntu  7.04
2006    SUSE    10.1
2006    Fedora  6
2008    Fedora  9
2006    Ubuntu  6.06
2008    Ubuntu  8.10
2006    Fedora  5
```

# Text Processing: join

- The **join** program joins lines of two files on a common field.
- It is similar to the JOIN operator on a database.
- To do so, let’s consider two files that have fields which are defined as keys.
- The customer file has three fields, a customer number, the first name and the last name.
- The orders file contains four fields, an order number, the customer number, the quantity and the item ordered.
- Both files share the customer number.

# Text Processing: join

- Performing a join operation would allows us to combine the fields in the two tables to achieve a useful result, such as an invoice.

```
pi@zeytin:~ $ cat customers.txt
1       Kaya    Oguz
2       Damla   Oguz
3       Ilker   Korkmaz
4       Umut    Avci
5       Coskun  Atay
6       Mehmet  Unluturk
pi@zeytin:~ $ cat orders.txt
35351   1       1       ANDROID TABLET
31233   2       1       KIKO
42344   3       1       ULTRABOOK
23421   3       2       EARPHONES
pi@zeytin:~ $ join -1 2 -2 1 orders.txt customers.txt
1 35351 1 ANDROID TABLET Kaya Oguz
2 31233 1 KIKO Damla Oguz
3 42344 1 ULTRABOOK Ilker Korkmaz
3 23421 2 EARPHONES Ilker Korkmaz
```

# Text Processing: join

- By default, join will use the first fields of the lines of files as the key values.

```
ilker@debian:~$ cat test.txt
1 ilker korkmaz 80
2 kaya oguz     85
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ cat test2.txt
1 ilker korkmaz 80
2 kaya oguz     58
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ join test.txt test2.txt
1 ilker korkmaz 80 ilker korkmaz 80
2 kaya oguz 85 kaya oguz 58
3 umut avci 85 umut avci 85
4 damla oguz 99 damla oguz 99
ilker@debian:~$ join -1 1 -2 1 test.txt test2.txt
1 ilker korkmaz 80 ilker korkmaz 80
2 kaya oguz 85 kaya oguz 58
3 umut avci 85 umut avci 85
4 damla oguz 99 damla oguz 99
```

# Text Processing: join

- By default, join expects that the files are sorted by the key values.

```
ilker@debian:~$ cat test.txt
1 ilker korkmaz 80
2 kaya oguz     85
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ cat test2.txt
1 ilker korkmaz 80
2 kaya oguz     58
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ join -1 1 -2 2 test.txt test2.txt
join: test2.txt:4: is not sorted: 4 damla oguz  99
ilker@debian:~$ join -1 2 -2 1 test.txt test2.txt
join: test.txt:4: is not sorted: 4 damla oguz   99
ilker@debian:~$ join -1 1 -2 4 test.txt test2.txt
join: test2.txt:2: is not sorted: 2 kaya oguz   58
ilker@debian:~$
```

# Text Processing: join

- By default, join expects that the files are sorted by the key values.

```
ilker@debian:~$ cat test.txt
1 ilker korkmaz 80
2 kaya oguz     85
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ cat test2.txt
1 ilker korkmaz 80
2 kaya oguz     58
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ join -1 1 -2 2 test.txt test2.txt
join: test2.txt:4: is not sorted: 4 damla oguz  99
ilker@debian:~$ join -1 2 -2 1 test.txt test2.txt
join: test.txt:4: is not sorted: 4 damla oguz   99
ilker@debian:~$ join -1 1 -2 4 test.txt test2.txt
join: test2.txt:2: is not sorted: 2 kaya oguz   58
ilker@debian:~$
```

# Text Processing: join

- The command join will give no output if there is no joined line based on key.

```
ilker@debian:~$ cat test.txt
1 ilker korkmaz 80
2 kaya oguz     85
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ cat test2.txt
1 ilker korkmaz 80
2 kaya oguz     58
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ join -1 4 -2 1 test.txt test2.txt
ilker@debian:~$ echo $?
0
ilker@debian:~$ join -1 2 -2 1 test.txt test2.txt
join: test.txt:4: is not sorted: 4 damla oguz   99
ilker@debian:~$ echo $?
1
ilker@debian:~$ join test.txt test2.txt
1 ilker korkmaz 80 ilker korkmaz 80
2 kaya oguz 85 kaya oguz 58
3 umut avci 85 umut avci 85
4 damla oguz 99 damla oguz 99
ilker@debian:~$ echo $?
0
```

# Text Processing: comm

- The **comm** program compares two sorted files line by line.
- It outputs three fields as the result of a line comparison.
- The first field contains lines unique to the first file, the second field to the second file, and the third field shows the lines shared by both files.
- We can suppress the fields using options -1, -2, -3, or a combination -12.

```
pi@zeytin:~ $ cat > f0
a
b
c
d
pi@zeytin:~ $ cat > f1
b
c
d
e
pi@zeytin:~ $ comm f0 f1
a
                    b
                    c
                    d
          e
pi@zeytin:~ $ comm -12 f0 f1
b
c
d
```

# Text Processing: comm

- Another example using **comm**

```
ilker@debian:~$ cat test.txt
1 ilker korkmaz 80
2 kaya oguz     85
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ cat test2.txt
1 ilker korkmaz 80
2 kaya oguz     58
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ comm test.txt test2.txt
                1 ilker korkmaz 80
        2 kaya oguz     58
2 kaya oguz     85
                3 umut avci     85
                4 damla oguz    99
ilker@debian:~$ comm -12 test.txt test2.txt
1 ilker korkmaz 80
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ comm -123 test.txt test2.txt
ilker@debian:~$ echo $?
0
ilker@debian:~$
```

# Text Processing: diff

- The **diff** program is one of the most common tools used by UNIX developers.
- Developers usually receive **patch** files for bug fixes.
- The diff and patch programs are used together.
- Let’s see how they work.

# Text Processing: diff

- **diff** has three types of output.
- The default one has the following change commands.
  - p1ap2, p1cp2, p1dp2
- Notice that, even though they are written in one piece, they contain three fields.
- The first field is the position in the first file (p1), the second is the command, and the third is the position in the second file (p2).

# Text Processing: diff

- **p1ap2** : The **a** is for the append command. Append the lines at the position p2 in the second file to the position p1 in the first file.
- **p1cp2** : The **c** is for change, or replace. Change the lines at the position p1 with the lines at the position p2 in the second file.
- **p1dp2** : The **d** is for delete. Delete the lines in the first file at position p1, which would have appeared at range p2 in the second file.
- It is also possible to use the context format with the option -c.

# Text Processing: diff

- The context format begins with the names of files and their timestamps.
- The first file is marked with asterisks, the second file with dashes.
- Then there are indicators for each line.

```
pi@zeytin:~ $ diff f0 f1
1d0
< a
4a4
> e
pi@zeytin:~ $ diff -c f0 f1
*** f0  2017-04-02 14:52:46.708540782 +0300
--- f1  2017-04-02 14:52:54.628463056 +0300
***************
*** 1,4 ****
- a
  b
  c
  d
--- 1,4 ----
  b
  c
  d
+ e
```

# Text Processing: diff

- If there are no indicators, there is no difference.
- A minus sign (-) indicates that a line is deleted.
- A plus sign (+) indicates that a line is added.
- An exclamation mark (!) indicates that a line is changed. Two versions of the line will be displayed, each in its respective section of the change group.

```
pi@zeytin:~ $ diff f0 f1
1d0
< a
4a4
> e
pi@zeytin:~ $ diff -c f0 f1
*** f0  2017-04-02 14:52:46.708540782 +0300
--- f1  2017-04-02 14:52:54.628463056 +0300
***************
*** 1,4 ****
- a
  b
  c
  d
--- 1,4 ----
  b
  c
  d
+ e
```

# Text Processing: diff

- The unified format, -u, is similar to context format but is more concise.
- Only the minus and plus signs are used to indicate the changes.
- If there is no change, no indicator is used.

```
pi@zeytin:~ $ diff -u f0 f1
--- f0  2017-04-02 14:52:46.708540782 +0300
+++ f1  2017-04-02 14:52:54.628463056 +0300
@@ -1,4 +1,4 @@
-a
 b
 c
 d
+e
```

# Text Processing: diff

- Another example using the command diff

```
ilker@debian:~$ cat test.txt
1 ilker korkmaz 80
2 kaya oguz     85
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ cat test2.txt
1 ilker korkmaz 80
2 kaya oguz     58
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ diff test.txt test2.txt
2c2
< 2 kaya oguz   85
---
> 2 kaya oguz   58
ilker@debian:~$
```

# Text Processing: diff

- Another example using the command diff

```
ilker@debian:~$ cat test.txt
1 ilker korkmaz 80
2 kaya oguz     85
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ cat test2.txt
1 ilker korkmaz 80
2 kaya oguz     58
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ diff -c test.txt test2.txt
*** test.txt    2019-04-05 08:43:36.752000000 +0300
--- test2.txt   2019-04-05 09:07:11.716000000 +0300
***************
*** 1,4 ****
  1 ilker korkmaz 80
! 2 kaya oguz   85
  3 umut avci   85
  4 damla oguz  99
--- 1,4 ----
  1 ilker korkmaz 80
! 2 kaya oguz   58
  3 umut avci   85
  4 damla oguz  99
```

# Text Processing: diff

- Another example using the command diff

```
ilker@debian:~$ cat test.txt
1 ilker korkmaz 80
2 kaya oguz     85
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ cat test2.txt
1 ilker korkmaz 80
2 kaya oguz     58
3 umut avci     85
4 damla oguz    99
ilker@debian:~$ diff -u test.txt test2.txt
--- test.txt    2019-04-05 08:43:36.752000000 +0300
+++ test2.txt   2019-04-05 09:07:11.716000000 +0300
@@ -1,4 +1,4 @@
 1 ilker korkmaz 80
-2 kaya oguz    85
+2 kaya oguz    58
 3 umut avci    85
 4 damla oguz   99
ilker@debian:~$
```

# Text Processing: diff and patch

- The diff file is useful to record the changes you make.
- This happens a lot if you are fixing bugs.
- Sometimes, you fix the bug, but you don’t have the write permissions on the source code management server, so you send a “patch”, which is simply the diff file which contains your changes.
- The developers then can apply these changes using the **patch** program.
- Let me first demonstrate using our f0 and f1.

# Text Processing: diff and patch

- What we do here is simply make the f0 the same as f1.
- We find the difference between them, save the output to a file.
- Then use this difference with patch, and the file f0 is patched.

```
pi@zeytin:~ $ diff -Naur f0 f1
--- f0  2017-04-02 14:52:46.708540782 +0300
+++ f1  2017-04-02 14:52:54.628463056 +0300
@@ -1,4 +1,4 @@
-a
 b
 c
 d
+e
pi@zeytin:~ $ diff -Naur f0 f1 > diff_file
pi@zeytin:~ $ patch < diff_file
patching file f0
pi@zeytin:~ $ cat f0
b
c
d
e
pi@zeytin:~ $ cat f1
b
c
d
e
```

# Text Processing: diff and patch

- Here is a sample from Linus Torvalds himself, available at https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=9e54ef9da519d9febfa61fa9392276fbfa6ce79c
- This commit, dated 2017.03.26, has a diff file that shows the changes made.

| | | |
|---|---|---|
| author | Linus Torvalds <torvalds@linux-foundation.org> | 2017-03-26 11:05:42 -0700 |
| committer | Linus Torvalds <torvalds@linux-foundation.org> | 2017-03-26 11:05:42 -0700 |
| commit | 9e54ef9da519d9febfa61fa9392276fbfa6ce79c (patch) | |
| tree | 6f3f4f227b039c18394a6ce053e9e99252afbdb3 | |
| parent | f1638fc65ee23875cf4021e342e8a564debe33e4 (diff) | |
| parent | 966fa72a716ceafc69de901a31f7cc1f52b35f81 (diff) | |

## Merge tag 'driver-core-4.11-rc4' of git://git.kernel.org/pub/scm/linux/kernel/git/gregkh/driver-core

```
Pull driver core fix from Greg KH:
 "Here is a single kernfs fix for 4.11-rc4 that resolves a reported
  issue.

  It has been in linux-next with no reported issues"

* tag 'driver-core-4.11-rc4' of git://git.kernel.org/pub/scm/linux/kernel/git/gregkh/driver-core:
  kernfs: Check KERNFS_HAS_RELEASE before calling kernfs_release_file()
```

**Diffstat**

-rw-r--r-- fs/kernfs/file.c 3

1 files changed, 2 insertions, 1 deletions

```
diff --git a/fs/kernfs/file.c b/fs/kernfs/file.c
index 8e4dc7a..ac2dfe0 100644
--- a/fs/kernfs/file.c
+++ b/fs/kernfs/file.c
@@ -809,7 +809,8 @@ void kernfs_drain_open_files(struct kernfs_node *kn)
                if (kn->flags & KERNFS_HAS_MMAP)
                        unmap_mapping_range(inode->i_mapping, 0, 0, 1);

-               kernfs_release_file(kn, of);
+               if (kn->flags & KERNFS_HAS_RELEASE)
+                       kernfs_release_file(kn, of);
        }

        mutex_unlock(&kernfs_open_file_mutex);
```

# Formatting Output: printf

- The **printf** command is a little different, as it is not used for pipelines.
- It works as

  printf “format” arguments

- The command is given a string containing a format description which is then applied to a list of arguments.
- It actually is very similar to its C counterpart.

```
pi@zeytin:~ $ printf "I formatted '%s' as a string.\n" foo
I formatted 'foo' as a string.
pi@zeytin:~ $ printf "An integer, %d, and a floating point %.2f\n" 12 33.3333
An integer, 12, and a floating point 33.33
```

# A video from the past

- The YouTube channel of AT&T has very interesting archive videos.
- One of them is titled UNIX: Making Computers Easier To Use, from 1982.
- You can watch Brian Kernighan give a very relaxed demonstration of the 1982 versions of these programs.
- The video also has significant people, such as Dennis Ritchie and Ken Thompson. Can you spot others?

# References


- William E. Shotts, Jr., The Linux Command Line
- UNIX: Making Computers Easier To Use -- AT&T Archives film from 1982, Bell Laboratories, online, https://www.youtube.com/watch?v=XvDZLjaCJuw